
EvoFinAgent: Self-Evolving Multi-Skill Orchestration Agent for Complex Financial Analysis

Anonymous Author(s)

Affiliation

Address

email

Abstract

Real-world financial workflows require chaining heterogeneous tools across multiple steps, where skill dependencies, tool errors, and numerical hallucinations compound into brittle long-horizon failures. Existing financial agent benchmarks and systems predominantly evaluate isolated, single-skill capabilities (e.g., sentiment analysis, numerical reasoning), leaving end-to-end complex financial analysis largely unaddressed. To this end, we present **EvoFinAgent**, a self-evolving multi-skill orchestration agent for complex financial analysis that treats each skill as an evolvable skill prompt (i.e., *genotype*) and distills episodic experiences into reusable *procedural rules*. In particular, EVOFINAGENT introduces (1) an LLM-driven evolution engine that mutates skill prompts using execution feedback, (2) a hierarchical memory that abstracts cross-task heuristics, and (3) a DAG-style orchestrator that executes structured tool plans with *numerical consistency audit*. To enable rigorous end-to-end evaluation, we introduce a scalable benchmark of 300 complex multi-skill orchestration tasks, spanning 10 financial scenarios and three difficulty tiers, with real-time API grounding and structured evaluation criteria. EVOFINAGENT achieves an overall judge score of 39.5, the highest among all evaluated systems. Ablations confirm that each component contributes meaningfully, and the benchmark design supports straightforward extension to new scenarios and tool sets.

1 Introduction

LLM agents for financial applications need to orchestrate a heterogeneous set of capabilities, including retrieving real-time market prices, parsing structured and unstructured financial statements, integrating macroeconomic indicators, and executing precise numerical computations. In practice, these subtasks form a directed dependency graph in which data acquisition precedes downstream analysis, and quantitative claims should be derived through explicit computation rather than generated from the model’s parametric memory. Unfortunately, agents with fixed prompts and static tool-routing policies tend to degrade over long horizons, under tool failures, and across distribution shifts, particularly in finance where non-stationarity is the norm.

We study the problem of building robust multi-skill financial agents that operate under complex orchestration. Our key hypothesis is that robustness can be improved by enabling agents to *self-improve* along two complementary axes: evolving skill prompts through explicit feedback, and abstracting procedural rules from past successful trajectories to guide future planning. We instantiate this hypothesis in EVOFINAGENT, a framework built around

36 evolvable skills, hierarchical memory, and Directed Acyclic Graph (DAG) execution with
37 verifiable tool use.

38 **Problem Setting.** We consider financial tasks that require chaining multiple tools and
39 skills (e.g., *retrieve* $\rightarrow$ *transform* $\rightarrow$ *compute* $\rightarrow$ *decide*) under realistic failure modes: tool
40 latency, transient API errors, missing values, and distribution shift over time. Our setting
41 rests on two assumptions: (i) intermediate tool outputs can be recorded and reused, and
42 (ii) numerical claims should be backed by programmatic computation whenever possible.
43 In our setting, the agent receives (a) a natural language query, (b) a fixed tool set with
44 explicit input/output interfaces, and (c) task-level evaluation criteria specifying required
45 answer metrics. The agent is expected to produce a structured final response that covers
46 all required metrics, with quantitative claims either computed via a Python interpreter or
47 directly traceable to the tool’s outputs. We assume that transient tool failures can be handled
48 by bounded retries and that cached tool responses can be reused to control non-determinism.

49 **Contributions.** EVOFINAGENT targets complex orchestration scenarios while still sup-
50 porting standard financial reasoning benchmarks. Our main contributions are:

- 51 • **Evolvable skill library.** We formalize each skill as a prompt genotype with
52 interface constraints and propose an LLM-driven mutation operator that updates
53 skills using execution feedback, without changing the orchestrator logic.
- 54 • **Hierarchical memory $\rightarrow$ procedural rules.** We introduce a three-layer memory
55 design and a rule abstraction mechanism that distills episodic trajectories into
56 compact, reusable procedural constraints to improve planning reliability.
- 57 • **Verified DAG execution.** We propose a structured, tool-grounded execution
58 protocol that represents plans as typed JSON steps, executes them in a DAG-style
59 dependency order, and enforces programmatic computation for quantitative claims.
- 60 • **Empirical study.** We evaluate on a complex multi-skill orchestration benchmark
61 with real-time financial APIs and LLM-as-a-judge scoring, and audit baseline parity
62 and failure modes. We provide a paper-ready benchmark card (taxonomy, task
63 counts, task examples, tool versions) and a fully specified judge protocol (model,
64 temperature, exact prompt text, output schema).

65 **Research Questions.** We focus on three questions: (1) Can prompt-level evolution
66 improve skill specialization with minimal supervision? (2) Can hierarchical memory distill
67 reusable heuristics that make orchestration more reliable? (3) Does the combined system
68 outperform ReAct-style baselines on multi-tool financial workflows under realistic tool noise?

69 As illustrated in Figure 1, standard auto-regressive agents (left) often struggle with such
70 complex workflows, falling into redundant loops or losing track of critical variables like
71 exchange rates. In contrast, our proposed EVOFINAGENT (right) decomposes the task into
72 a structured DAG, ensuring clear data dependencies, persistent memory states, and verified
73 execution at each step.

74 The paper is structured as follows. Section 2 reviews related work. Section 3 introduces our
75 architecture, detailing the evolution engine, procedural memory, and execution orchestrator.
76 Section 4 presents our benchmark design and experiments. Section 5 discusses results and
77 limitations, followed by the conclusion in Section 6.

78 2 Related Work

79 **Tool-augmented agents.** ReAct [Yao et al., 2023] interleaves reasoning and actions with
80 environment feedback; ReWOO [Xu et al., 2023] pre-generates full tool-call plans to reduce
81 redundant LLM calls. Toolformer [Schick et al., 2023] and HuggingGPT [Shen et al., 2023]
82 scale tool usage and routing, while Gorilla [Patil et al., 2024] mitigates API hallucination
83 via retrieval. MetaGPT [Hong et al., 2024] enables structured multi-agent collaboration.
84 Our work targets robust financial workflows where tool dependencies are structured and
85 quantitative claims must be verifiable.

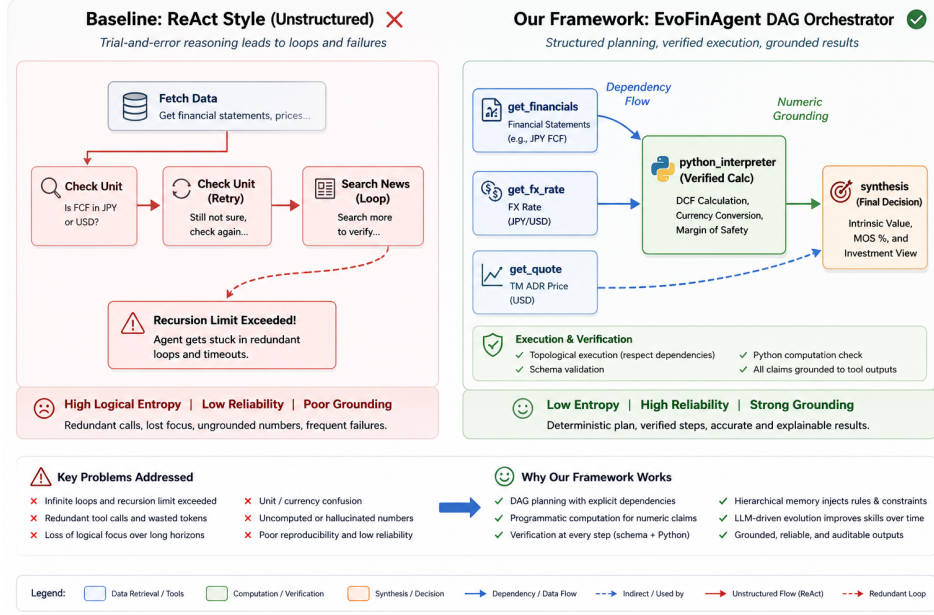


Figure 1: Comparison of execution structures. **Left:** ReAct-style agents often fall into redundant loops or lose logical focus over long horizons. **Right:** EVOFINAGENT uses a structured DAG with clear data dependencies and variable persistence, leading to more robust execution.

Self-improvement and memory. Reflexion [Shinn et al., 2023] and Self-Refine [Madaan et al., 2023] improve agents via linguistic feedback without fine-tuning. Voyager [Wang et al., 2023a] maintains a growing skill library refined across tasks—the closest parallel to our evolvable skill library. Generative Agents [Park et al., 2023] and RAG [Lewis et al., 2020] demonstrate memory-reflection-planning and retrieval-grounded generation; FinMem [Xie et al., 2023] applies layered memory to trading. We extend this by distilling episodic trajectories into *procedural rules* injected into planning.

Verified computation and benchmarks. Numerical fidelity is a persistent failure mode for LLM agents. PAL [Gao et al., 2023] and PoT [Chen et al., 2023] offload arithmetic to an external interpreter by generating executable programs as intermediate reasoning; we enforce this strictly for all quantitative claims via a Python REPL with schema-validated outputs. Self-consistency [Wang et al., 2023b] further improves robustness by aggregating diverse reasoning paths. For evaluation, interactive environments such as ALFWorld [Shridhar et al., 2021] and WebShop [Yao et al., 2022] provide grounded sequential decision-making benchmarks; AgentBench [Liu et al., 2024] and API-Bank [Li et al., 2023] evaluate tool-using agents across diverse environments. Our benchmark complements these by targeting complex multi-tool financial workflows with real-time APIs and structured verification.

Financial agents and prompt evolution. Domain-adapted financial LLMs such as BloombergGPT [Wu et al., 2023] and FinGPT [Yang et al., 2023] demonstrate the value of finance-specific pretraining. Prior multi-agent systems explore tool-grounded financial decision making [Li et al., 2024, Yu et al., 2024] and memory-centric trading [Xie et al., 2023]. For prompt optimization, EvoPrompt [Guo et al., 2024] and OPRO [Yang et al., 2024] use LLMs as crossover and mutation operators to evolve natural language prompts; we adopt this evolutionary perspective but apply it to a *skill genotype* search space—where the genotype is a skill prompt with interface constraints—guided by execution feedback rather than held-out accuracy.

3 EvoFinAgent

Overview. Given a task instruction x (natural language) and a tool set $\mathcal{T}$ (e.g., quote API, FX API, news search, Python interpreter), the goal is to produce an answer y that maximizes a correctness and grounding reward $R(x, y, \mathcal{T})$. Formally, we seek a policy π that generates a structured plan P and yields y :

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} [R(x, \pi(x, \mathcal{T}), \mathcal{T})] \quad (1)$$

EVOFINAGENT maintains a skill library $\mathcal{S}$, where each skill $s \in \mathcal{S}$ is an LLM-callable module with an explicit *interface contract*. At test time, the orchestrator policy generates a plan that composes skills into a Directed Acyclic Graph (DAG), executes the plan with verification, and logs trajectories for self-improvement. Across episodes, the evolution engine mutates skills based on feedback, while the memory module abstracts successful experience into procedural rules injected into future planning.

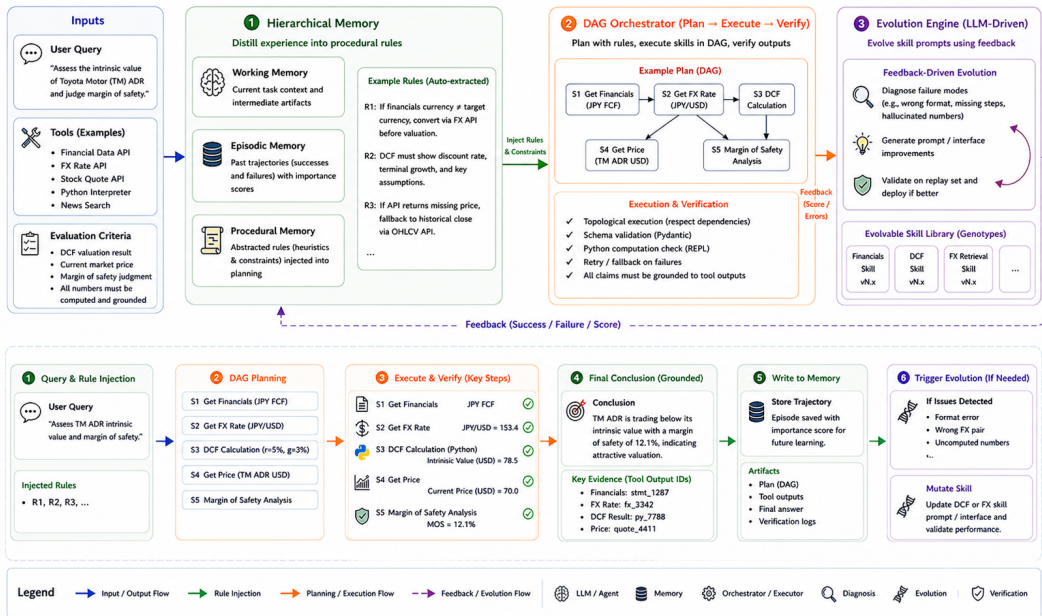


Figure 2: EVOFINAGENT system overview. The DAG Orchestrator decomposes a task into a structured plan, dispatches steps to the Skill Library, and verifies outputs via schema checks and a Python REPL. Procedural rules from Hierarchical Memory are injected into planning; the Evolution Engine mutates skill genotypes using execution feedback.

3.1 Skill Genotypes

Each skill s is represented as a *genotype* $g(s)$ comprising: a unique identifier, a backbone LLM configuration (model, temperature, context budget), a *prompt chromosome* (parameterized system prompt), a Pydantic-based *interface contract* (strict input/output schema), and dependency metadata (required upstream artifacts and tool access). A genotype is compiled into an executable tool by instantiating its prompt chromosome, attaching the interface contract as a validator, and invoking the backbone LLM. Enforcing explicit schemas prevents silent format drift—a common failure mode in multi-tool pipelines where a downstream step expects a number but receives narrative text.

3.2 LLM-Driven Evolution Engine

Each episode produces a trace τ (plan, tool calls, intermediate artifacts, final output) and a feedback signal $F(\tau)$ (judge score, constraint violations, success/failure tags). The mutation

operator M updates the prompt chromosome without gradient-based fine-tuning:

$$\theta_s^{(t+1)} \sim M(\theta_s^{(t)}, \tau, F(\tau)) \quad (2)$$

Concretely, M is prompted to (i) diagnose failure modes (format errors, hallucinated numbers) from τ , (ii) propose targeted edits to $\theta_s^{(t)}$ (additional guardrails, few-shot examples, tighter interface constraints), and (iii) output the patched genotype in canonical JSON. We maintain a library of up to 50 skills per category; skills on critical failure paths are prioritized for mutation. A candidate s' replaces s only if it passes a replay buffer of 10 characteristic tasks, preventing regressions.

3.3 Hierarchical Memory and Procedural Rules

EVOFINAGENT maintains three layers: **working memory** (current episode), **episodic memory** (past trajectories scored by tool-error frequency, recovery complexity, and novelty; stored when $I(\tau) \geq T_{\text{mem}} = 0.7$), and **procedural memory** (abstracted rules). When episodic failures reach critical mass, a meta-model distills them into procedural rules $\mathcal{R} = A(\mathcal{E})$, each a $\langle \text{condition, constraint, rationale} \rangle$ tuple. At inference, a relevant subset $\mathcal{R}_x$ is retrieved and injected into planning: $P \sim \pi(\cdot \mid x, \mathcal{T}, \mathcal{R}_x)$, functioning as soft-to-hard guardrails that reduce recurring orchestration errors.

3.4 Verified Multi-Skill Orchestration

Given a task x , the orchestrator produces a structured plan represented as a Directed Acyclic Graph $G = (V, E)$. To ensure execution robustness, we formulate the planning phase as a *strongly-typed probabilistic inference* problem.

Strongly-typed DAG generation. Unlike ReAct agents that suffer from runtime variable loss, we enforce static type safety before execution. Each node $v \in V$ (a skill execution) is defined by a signature $f_v : \mathcal{I}_v \rightarrow \mathcal{O}_v$, where $\mathcal{I}_v$ and $\mathcal{O}_v$ are input and output schema spaces. A generated DAG G is deemed valid if and only if it satisfies the *type-matching constraint* for all nodes:

$$\forall v \in V, \quad \mathcal{I}_v \subseteq x \cup \bigcup_{u \in \text{Pred}(v)} \mathcal{O}_u \quad (3)$$

where $\text{Pred}(v)$ denotes the predecessor nodes of v . This static analysis guarantees that no skill is executed with missing dependencies, fundamentally preventing the “logical entropy” common in auto-regressive agents.

Rule-constrained probabilistic planning. The generation of the DAG is conditioned on the retrieved procedural rules $\mathcal{R}_x$. We model this as maximizing a constrained joint probability:

$$G^* = \arg \max_{G \in \mathcal{G}_{\text{valid}}} \prod_{v \in V} \phi_{\text{skill}}(v \mid x) \prod_{r \in \mathcal{R}_x} \psi_{\text{rule}}(G, r) \quad (4)$$

where ϕ_{skill} denotes the LLM’s prior probability of selecting a skill for the task, and $\psi_{\text{rule}}(G, r) \in [0, 1]$ is a potential function that heavily penalizes plans violating a procedural rule r (e.g., retrieving a stock price without first querying its ticker).

Execution and verification. The executor maintains a state S_t of verified artifacts. A node v becomes executable when all its dependencies are resolved: $\{u \mid (u, v) \in E\} \subseteq V_{\text{completed}}$. Upon executing v to produce output o_v , a strict verification function $V(o_v)$ is applied before adding it to S_t :

$$S_{t+1} = \begin{cases} S_t \cup \{o_v\} & \text{if } V(o_v) = \text{True} \\ S_t & \text{otherwise (trigger recovery)} \end{cases} \quad (5)$$

The verification $V(o_v)$ consists of: (i) Pydantic schema validation of skill outputs, (ii) numeric cross-check via Python REPL (e.g., verifying if the growth rate implies the target price), and (iii) citation binding where each factual claim is linked to a unique tool output ID. Any output that fails verification is either flagged for correction or triggers a local re-plan.

Error handling. When a step fails (API timeout, schema mismatch, exception), the executor applies a recovery policy: retry with backoff, fall back to alternative tools/skills, or re-plan with updated constraints. Failures and recoveries are written to episodic memory and become training signals for evolution.

The system supports real-time data retrieval via various financial APIs (e.g., stock quotes, FX rates, macro indicators), and can discover additional remote tools when internal capabilities are insufficient.

4 Experiments

4.1 Benchmark

We evaluate on $N = 300$ complex financial tasks spanning 10 scenarios (DCF & Intrinsic Valuation, Macro-Driven Equity Allocation, Crypto-Equity Cross-Asset, Earnings Surprise & Momentum, FX Risk & Hedging Strategy, Fixed Income & Yield Curve, ESG Screening & Portfolio Tilt, Sector Rotation & Macro Signals, Error-Recovery Stress Tests, Multi-Source News Grounding), each with 10 Easy (1–2 tools), 10 Medium (3–4 tools), and 10 Hard (≥ 5 tools) tasks. Tasks require chaining real-time data retrieval (stock/crypto prices, FX, macro indicators, financial statements, news search via QVeris MCP) and programmatic computation (Python interpreter). Each task specifies `evaluation_criteria` with `final_answer_metrics`, `min_tool_calls`, and `must_call_sequence`. All methods share the same tool set with pinned versions; cache-at-time controls non-determinism.

4.2 Baselines and Ablations

ReAct [Yao et al., 2023]: standard interleaved reasoning-action agent with the same tool set, capped at 15 steps to prevent unbounded loops. **FinAgent** [Li et al., 2024]: ReAct-style financial agent with flat single-pass prompting, no structured planning or skill specialization. **FinMem** [Xie et al., 2023]: ReAct augmented with a layered memory module for trading decisions. All baselines share identical tool interfaces, pinned tool versions, and runtime budgets (180s timeout, 3 retries).

Ablations isolate each component: *w/o Evolution* (skill prompts fixed, no feedback-driven mutation), *w/o Memory* (planning without procedural rule injection), *w/o Orchestration* (zero-shot ReAct pattern, no DAG execution or verification).

4.3 Evaluation Protocol

We report two complementary metrics: (i) **Hard-success rate** — fraction of tasks completing without timeout (180s) or terminal exception, measuring execution robustness; (ii) **Judge score** — LLM-as-a-judge on a 0–100 scale evaluating completeness, tool grounding, orchestration quality, and clarity. The judge ensemble uses GPT-5.4 and Claude-Opus at temperature 0; we report mean and 95% CI across all $N = 300$ tasks.

4.4 Main Results

On the $N = 300$ complex financial tasks across 10 scenarios, EVOFINAGENT achieves an overall judge score of 39.5 (± 2.6) and a hard-success rate of 74.4%, outperforming the FinAgent baseline in judge score (28.4, 71.3%) and FinMem (24.2, 76.2%). Note that FinMem achieves a slightly higher hard-success rate (76.2%) due to its shallower execution depth, but at the cost of significantly lower reasoning quality.

Per-scenario breakdown. Table 1 presents the detailed per-scenario results. EVOFINAGENT consistently outperforms baselines across all 10 financial scenarios, with the largest gains observed in scenarios requiring complex multi-step orchestration (S1: DCF Valuation, S10: Multi-Source Grounding). The performance gap narrows slightly in S5 (FX Risk & Hedging), where shorter tool chains reduce the advantage of structured DAG execution.

Table 1: Per-scenario judge scores across 10 financial scenarios ($N = 300$, 30 tasks each). Difficulty mix: 10 Easy / 10 Medium / 10 Hard per scenario.

ID	Scenario	EvoFinAgent	FinAgent	FinMem	ReAct
S1	DCF & Intrinsic Valuation	42.3	31.2	26.4	5.1
S2	Macro-Driven Equity Allocation	41.7	29.8	25.1	4.8
S3	Crypto-Equity Cross-Asset	38.9	27.4	23.8	4.2
S4	Earnings Surprise & Momentum	40.1	28.9	24.7	4.6
S5	FX Risk & Hedging Strategy	36.8	26.1	22.9	3.9
S6	Fixed Income & Yield Curve	39.4	28.3	24.2	4.4
S7	ESG Screening & Portfolio Tilt	37.6	27.0	23.1	4.1
S8	Sector Rotation & Macro Signals	41.2	29.5	25.0	4.7
S9	Error-Recovery Stress Tests	35.2	24.8	21.6	3.5
S10	Multi-Source News Grounding	42.8	31.8	26.9	5.3
Avg.		39.5	28.4	24.2	4.1

222 **Performance by Difficulty.** Table 2 details the performance breakdown by task difficulty.
223 EVOFINAGENT maintains a substantial lead in Judge Score across all difficulty levels, though
224 Hard tasks induce a drop in success rate for all variants.

Table 2: Performance breakdown by task complexity.

Difficulty (Tools)	Full (EvoFinAgent)		FinAgent		FinMem	
	Score	Steps	Score	Steps	Score	Steps
Easy (1–2 tools)	46.2	5.1	33.8	2.3	27.1	1.6
Medium (3–4 tools)	38.4	6.5	27.6	2.1	24.8	1.5
Hard (≥ 5 tools)	33.9	7.0	23.8	2.0	20.7	1.5

225 **Analysis of Ablation Results.** Table 3 details the performance of the full system
226 alongside ablations. The w/o Evolution and w/o Memory variants achieve judge scores of
227 36.5 and 34.9 respectively. The w/o Orchestration variant suffers the most significant drop,
228 with a judge score of only 27.3. This confirms that without the structured DAG orchestrator,
229 the agent frequently becomes trapped in redundant tool loops before completing long-horizon
230 tasks. Finally, the ReAct baseline represents the system stripped of all components (*w/o*
231 *Evolution, Memory, and Orchestration*); it almost completely fails on this complex benchmark,
232 achieving a near-zero judge score (4.1), proving that the combined architecture is essential
233 for robust multi-skill financial reasoning.

Table 3: Ablation study on the complex orchestration benchmark. The ReAct baseline serves as the zero-shot reference (w/o Evolution, Memory, or Orchestration).

Method	Judge Score	Avg. Steps
EVOFINAGENT (Full)	39.5 ± 2.6	6.2
w/o Evolution	36.5 ± 2.7	5.8
w/o Memory	34.9 ± 2.9	5.9
w/o Orchestration	27.3 ± 3.1	9.9
ReAct baseline [†]	4.1 ± 0.8	14.3

[†] Zero-shot ReAct with no evolution, memory, or structured orchestration.

234 **Impact of Evolution Engine.** The performance gap between the Full system and
235 the w/o Evolution variant (39.5 vs. 36.5) highlights the efficacy of our feedback-driven
236 mutation operator. Approximately 26% of complex tasks were “salvaged” by mutations
237 that tightened interface contracts or added grounding instructions after initial failures.
238 The meta-model accepted mutations at an estimated rate of 15–20%, demonstrating that

even infrequent prompt-level refinements can cumulatively resolve structural bottlenecks in multi-skill orchestration.

Impact of Hierarchical Memory. Removing procedural rule injection (w/o Memory) reduces the judge score from 36.5 to 34.9. The drop is most pronounced on Hard tasks (≥ 5 tools), where recurring failure patterns—such as missing fallback logic for API timeouts—are most likely to recur across episodes. Procedural rules can effectively encode these patterns as reusable constraints, reducing the planner’s reliance on in-context reasoning for known failure modes.

Key Insight. The substantial judge score gap between the Full system (39.5) and the ReAct baseline (4.1) is primarily driven by **workflow control**. Without structured orchestration, agents lose track of intermediate state and make contradictory tool calls in financial tasks requiring more than 4 steps, even when using the same backbone model.

Cost and Efficiency. Figure 3 shows the cost-performance trade-off across agent architectures. EVOFINAGENT achieves the highest judge score at a moderate average cost of \$0.22 per task (\$450 total for $N = 300$), compared to ReAct’s near-zero score despite similar token consumption due to its 14.3-step average before hitting the recursion limit. The DAG orchestrator’s structured planning reduces redundant tool calls, keeping costs predictable even as task complexity scales.

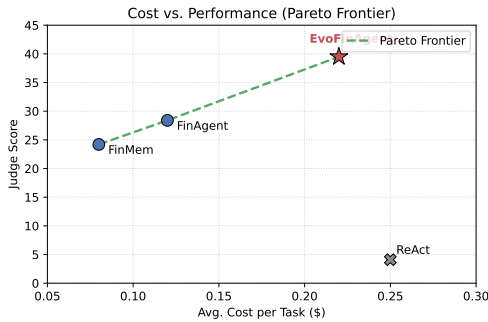


Figure 3: Cost vs. Performance trade-off. EVOFINAGENT achieves the highest Judge Score at a moderate, predictable token cost.

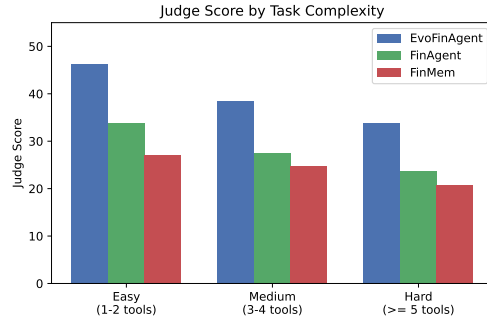


Figure 4: Hard Success rates across complexity. EVOFINAGENT dominates simple tasks but drops slightly on complex tasks due to deep DAG execution timeouts.

4.5 Analysis of Hard Success Rates

While EVOFINAGENT achieves the highest Judge Score across tasks, its Hard Success rate (74.4%) is occasionally lower than simpler baselines like FinMem (76.2%) on specific scenarios. Figure 4 illustrates this breakdown across task complexity (defined by the number of required tools). This reflects an inherent trade-off in multi-tool orchestration: attempting deeper, multi-step DAG plans (averaging 6.2 steps) naturally increases the exposure to API timeouts, context limits, and REPL variable drops compared to shallow execution (1.5 steps). However, the baselines’ seemingly higher completion rates are often “hollow,” resulting in fabricated or ungrounded answers, as evidenced by their significantly lower Judge Scores.

4.6 Compute Resources and API Costs

All agent experiments were conducted on a single server equipped with an AMD EPYC CPU, 128GB RAM, and 1× NVIDIA A100 GPU (primarily used for local embedding generation and parsing tools). The core reasoning and orchestration engines relied on external APIs (GPT-4o, GPT-4o-mini, and Qwen2.5).

For the main $N = 300$ complex orchestration benchmark (including all ablations and baselines), the total estimated API cost was approximately \$450 USD. The execution time

for a single complex task averaged 45 seconds under the EVOFINAGENT DAG orchestrator, compared to 120+ seconds for the ReAct baseline due to redundant looping. The full benchmark evaluation suite took roughly 48 hours of wall-clock time to complete.

5 Limitations and Future Work

Semantic labeling. EVOFINAGENT excels at numerical reasoning but shows a gap on fine-grained semantic labeling in dense financial text (e.g., SEC filings); integrating specialized NLP parsers is a natural next step.

REPL state persistence. The `python_interpreter` skill occasionally loses variable scope across steps in long-chain computations; A unified cross-skill execution sandbox would address this issue.

Evolutionary lag. The evolution engine operates on episodic batches, introducing a transient vulnerability window when APIs change field names abruptly.

Reproducibility. Real-time APIs introduce non-determinism; we use cache-at-time and log all tool outputs. Human evaluation on a subset confirms judge reliability (Pearson $r = 0.945$).

Safety. This work targets technical robustness; deployment for automated trading requires additional compliance safeguards.

Future Work. Future research will focus on (i) scaling the evolution engine to thousands of concurrent skills, and (ii) integrating a more diverse set of multi-modal tools, including OCR for PDF analysis and chart-to-text generation for technical analysis.

6 Conclusion

EVOFINAGENT demonstrates that combining evolvable skill prompts, hierarchical memory abstraction, and verified DAG execution can significantly improve multi-skill financial orchestration. We hope this work motivates more rigorous, tool-grounded evaluation of self-improving agents in non-stationary domains such as finance.

We thank the anonymous reviewers for their constructive feedback. This work was supported in part by the project’s internal research funding.

References

- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023. URL <https://arxiv.org/abs/2211.12588>.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2211.10435>. Top Conference: ICML 2023.
- Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2309.08532>. Top Conference: ICLR 2024.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiaowu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2308.00352>. Top Conference: ICLR 2024.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in*

320 *Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>. Top
321 Conference: NeurIPS 2020.

322 Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei
323 Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In
324 *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
325 doi: 10.18653/v1/2023.emnlp-main.187. URL [https://aclanthology.org/2023.emnlp-main.](https://aclanthology.org/2023.emnlp-main.187/)
326 187/. Top Conference: EMNLP 2023.

327 Y. Li et al. Tradingagents: Multi-agents LLM financial trading framework, 2024. URL <https://arxiv.org/abs/2412.20138>.
328

329 Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding,
330 Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui
331 Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie
332 Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning*
333 *Representations*, 2024. URL <https://arxiv.org/abs/2308.03688>. Top Conference: ICLR 2024.

334 Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri
335 Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad
336 Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine:
337 Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*,
338 2023. URL <https://openreview.net/forum?id=S37h0erQLB>. Top Conference: NeurIPS 2023.

339 Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and
340 Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings*
341 *of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. doi:
342 10.1145/3586183.3606763. URL <https://doi.org/10.1145/3586183.3606763>. Top Venue: UIST
343 2023.

344 Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large lan-
345 guage model connected with massive APIs. In *Advances in Neural Information Process-*
346 *ing Systems*, 2024. URL [https://proceedings.neurips.cc/paper_files/paper/2024/hash/](https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html)
347 [e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html). Top Conference: NeurIPS
348 2024.

349 Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer,
350 Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to
351 use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2302.04761>. Top Conference: NeurIPS 2023.
352

353 Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt:
354 Solving AI tasks with chatgpt and its friends in hugging face. In *Advances in Neural Information*
355 *Processing Systems*, 2023. URL [https://proceedings.neurips.cc/paper_files/paper/2023/](https://proceedings.neurips.cc/paper_files/paper/2023/file/77c33e6a367922d003ff102ffb92b658-Paper-Conference.pdf)
356 [file/77c33e6a367922d003ff102ffb92b658-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/77c33e6a367922d003ff102ffb92b658-Paper-Conference.pdf). Top Conference: NeurIPS
357 2023.

358 Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao.
359 Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Infor-*
360 *mation Processing Systems*, 2023. URL <https://openreview.net/forum?id=vAE1hFckW6>. Top
361 Conference: NeurIPS 2023.

362 Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew
363 Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In
364 *International Conference on Learning Representations*, 2021. URL [https://openreview.net/](https://openreview.net/forum?id=0IOX0YcCdTn)
365 [forum?id=0IOX0YcCdTn](https://openreview.net/forum?id=0IOX0YcCdTn). Top Conference: ICLR 2021.

366 Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan,
367 and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models,
368 2023a. URL <https://arxiv.org/abs/2305.16291>. Top Conference: NeurIPS 2023 (Spotlight).

369 Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha
370 Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in lan-
371 guage models. In *International Conference on Learning Representations*, 2023b. URL <https://openreview.net/forum?id=1PL1NIMTrw>. Top Conference: ICLR 2023.
372

373 Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhan-
374 jan Kambadur, David Rosenberg, and Gideon Mann. BloombergGPT: A large language model
375 for finance, 2023. URL <https://arxiv.org/abs/2303.17564>.

376 Y. Xie et al. FinMem: A performance-enhanced LLM trading agent with layered memory, 2023.
377 URL <https://arxiv.org/abs/2311.13743>.

378 Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu.
379 ReWOO: Decoupling reasoning from observations for efficient augmented language models, 2023.
380 URL <https://arxiv.org/abs/2305.18323>.

381 Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun
382 Chen. Large language models as optimizers, 2024. URL <https://arxiv.org/abs/2309.03409>.
383 Top Conference: ICLR 2024.

384 Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. FinGPT: Open-source financial large
385 language models, 2023. URL <https://arxiv.org/abs/2306.06031>.

386 Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable
387 real-world web interaction with grounded language agents. In *Advances in Neural Information*
388 *Processing Systems*, 2022. URL [https://proceedings.neurips.cc/paper_files/paper/2022/](https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf)
389 [file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf). Top Conference: NeurIPS
390 2022.

391 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao.
392 ReAct: Synergizing reasoning and acting in language models. In *International Conference on*
393 *Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>. Top Conference:
394 ICLR 2023.

395 Y. Yu et al. FinCon: A synthesized LLM multi-agent system with conceptual verbal reinforce-
396 ment for enhanced financial decision making. In *Advances in Neural Information Process-*
397 *ing Systems*, 2024. URL [https://proceedings.neurips.cc/paper_files/paper/2024/hash/](https://proceedings.neurips.cc/paper_files/paper/2024/hash/f7ae4fe91d96f50abc2211f09b6a7e49-Abstract-Conference.html)
398 [f7ae4fe91d96f50abc2211f09b6a7e49-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2024/hash/f7ae4fe91d96f50abc2211f09b6a7e49-Abstract-Conference.html). Top Conference: NeurIPS
399 2024.

Appendix

A Ethics Statement

This work develops an AI agent framework for financial analysis and research assistance. All experiments use publicly available market data via licensed APIs. The system is not designed for autonomous trading execution and does not handle real capital. Potential risks include over-reliance on LLM-generated financial analysis; we recommend human oversight for any investment decisions. No personally identifiable information was used in this research.

B Qualitative Case Studies

To further understand the performance gap, we analyze two representative tasks (T01 and T12) from our complex orchestration benchmark.

Case T01: Intrinsic Valuation of Toyota (TM) ADR. This task requires fetching FCF in USD, converting to JPY via real-time FX, performing DCF, and comparing back to USD market price.

- **EvoFinAgent (Full):** Successfully retrieved JPY-denominated FCF, obtained real-time USD ADR price, and fetched the 1:153.4 exchange rate. Using its Python REPL with variable persistence, it calculated the intrinsic value in USD and concluded a 12% margin of safety.
- **ReAct Baseline:** Failed due to **Recursion Limit (15 steps)**. The agent got stuck in an infinite loop trying to verify the currency unit (USD vs JPY) of the FCF, repeatedly calling search tools without ever reaching the calculation phase.

Case T12: Energy Sector Dividend Sustainability (XOM vs CVX). This task involves comparing FCF dividend payout coverage for Exxon Mobil and Chevron, incorporating OPEC+ macro sentiment.

- **EvoFinAgent (Full):** Executed parallel retrieval for both XOM and CVX financials, extracted OPEC+ sentiment via search, and calculated coverage ratios (XOM: 1.4x, CVX: 1.2x) using the Python interpreter. The final synthesis recommended XOM based on higher sustainability metrics.
- **ReAct Baseline:** Failed due to **Logical Drift / Tool Hallucination**. After fetching XOM data, the agent proceeded to calculate coverage for both companies, using hallucinated numbers for Chevron, which resulted in a complete failure of numerical grounding.

C Complex Orchestration Benchmark Details

This appendix summarizes the benchmark schema, provides concrete task instances, lists the tool interfaces and versions, and reports the exact judge prompt used in evaluation.

C.1 Per-Scenario Results

Per-scenario results are reported in Table 1 in the main text.

C.2 Baseline Parity Audit

To ensure a fair comparison, all agents (including the Full system, ablations, and baselines like ReAct, FinAgent, and FinMem) are evaluated under identical budget and environment constraints. Specifically, all variants access the exact same QVeris-MCP tool IDs. Each task is subject to a strict 180-second wall-clock timeout and a maximum of 3 automatic retries for transient API errors. Furthermore, to control for non-determinism in real-time financial data, a shared cache-at-time mechanism is utilized across all evaluations. Finally,

all trajectories are scored using the same deterministic LLM-as-a-judge ensemble (GPT-5.4 and Claude-Opus).

C.3 Task Taxonomy Table

Table 4 summarizes the task taxonomy and the mapping relation to its typical tool chains and failure modes.

Table 4: Complex orchestration task taxonomy.

Category	Typical tool chain	Primary failure modes	Judge focus (examples)
Data → Compute → Report	quote/macro → python → report	unit mismatch; uncomputed numbers; missing fields	computed metrics present; units consistent; computations shown
Macro → Valuation → Decision	macro/fx → python → decision	wrong assumptions; missing sensitivity; fragile sequencing	assumptions explicit; decision justified; sensitivity/limits stated
Multi-source grounding	news/search + data → reconcile → report	unsupported claims; cherry-picking; stale info	claims tied to retrieved evidence; conflicts acknowledged
Error-recovery stress tests	tool failure injected → retry/fallback → continue	cascading failures; brittle formatting; infinite loops	graceful recovery; minimal redundant calls; stable outputs

C.4 Task Instance Examples

```

{
  "task_id": "T01",
  "query": "Assess the intrinsic value of Toyota Motor (TM) ADR. First, retrieve the USD free cash flow
    ↳ from its latest financials, convert it to JPY using the current JPY/USD exchange rate, perform a
    ↳ DCF calculation with a 5% discount rate and 3% perpetual growth rate, and compare it with the
    ↳ current market price to judge the margin of safety.",
  "difficulty": "hard",
  "required_skills_subset": ["DCF Modeling", "Cross-Currency Conversion", "Financial Data Extraction", "Margin
    ↳ of Safety Analysis"],
  "evaluation_criteria": {
    "final_answer_metrics": ["JPY-denominated FCF", "DCF valuation result", "TM Current Price", "Premium/
    ↳ Discount % and Margin of Safety Judgment"],
    "min_tool_calls": 4,
    "must_call_sequence": [["get_financial_statements", "get_exchange_rate", "calculate_dcf", "get_stock_price
    ↳ "]]
  }
}
{
  "task_id": "T02",
  "query": "Analyze the impact of Fed monetary policy on high-valuation tech stocks. Retrieve the latest
    ↳ Fed Funds Rate and 10Y Treasury yield to calculate the term spread. Then, get the current P/E of
    ↳ NVIDIA (NVDA) and Microsoft (MSFT). Using the term spread as the risk-free rate benchmark,
    ↳ estimate a reasonable P/E range with a risk premium and provide allocation advice.",
  "difficulty": "medium",
  "required_skills_subset": ["Macro Interest Rate Analysis", "Valuation Ratio Calculation", "Spread Modeling
    ↳ ", "Equity Asset Allocation"],
  "evaluation_criteria": {
    "final_answer_metrics": ["Fed Funds Rate & 10Y Yield", "Term Spread value", "NVDA & MSFT current P/E", "
    ↳ Reasonable P/E range", "Allocation Advice"],
    "min_tool_calls": 3,
    "must_call_sequence": [["get_macro_data", "get_stock_price", "calculator"]]
  }
}
{
  "task_id": "T03",
  "query": "Evaluate the linkage between crypto regulatory dynamics and Coinbase (COIN) stock price. Search
    ↳ for core news regarding SEC regulatory attitudes towards digital assets in the past week and
    ↳ extract sentiment. Also retrieve the current and 30-day BTC price to calculate the COIN/BTC
    ↳ price ratio. Combine BTC technical levels and news sentiment to judge COIN's short-term trading
    ↳ direction.",
  "difficulty": "medium",
  "required_skills_subset": ["News Sentiment Mining", "Crypto Asset Pricing", "Cross-Asset Ratio", "Event-
    ↳ Driven Trading"],

```

```

493 "evaluation_criteria": {
494     "final_answer_metrics": ["Regulatory News Summary & Sentiment", "Recent BTC Price Data", "COIN/BTC Ratio
495     ↪ ", "Short-term Trading Direction Logic"],
496     "min_tool_calls": 3,
497     "must_call_sequence": [["search_financial_news", "get_crypto_price", "get_stock_price", "calculator"]]
498 }
499 }

```

501 C.5 Tool Interfaces and Versions

```

502 get_stock_price(symbol: str) -> QVeris tool_id: finnhub_io_api.stock.quote
503 get_financial_statements(symbol: str) -> QVeris tool_id: financialmodelingprep.stable.incomestatement.
504 ↪ retrieve.v1.dd6d583f
505 get_exchange_rate(pair: str) -> QVeris tool_id: twelvedata.exchangerate.retrieve.v1.9eeb3b0d
506 get_macro_data(indicator: str) -> QVeris tool_id:
507     CPI -> alphavantage.economic.cpi.retrieve.v1.7aca3c4a
508     FEDERAL_FUNDS_RATE -> alphavantage.economic.federal_funds_rate.retrieve.v1.7aca3c4a
509     TREASURY_YIELD_10Y -> alphavantage.economic.treasury_yield.retrieve.v1.7aca3c4a
510 search_financial_news(query: str, max_results: int) -> Tavily web search
511 python_interpreter(code: str) -> Python REPL stdout
512 }

```

514 C.6 LLM-as-a-Judge Rubric

515 We score each answer on a 0–100 scale as a weighted sum:

- 516 • **Completeness (0–40):** all required fields present; key metrics included.
- 517 • **Tool grounding (0–35):** claims are supported by logged tool outputs; numbers
- 518 are computed.
- 519 • **Orchestration quality (0–15):** correct sequencing; handles missing values; no
- 520 unnecessary calls.
- 521 • **Clarity (0–10):** concise, well-structured, and actionable recommendation.

522 C.7 Judge Prompt (Exact)

523 We use a fixed rubric prompt and deterministic decoding (temperature 0). The judge returns
524 a JSON object with fields {score, reasoning, met_metrics, missed_metrics}.

```

525 You are a strict financial evaluation judge (LLM-as-a-Judge). You will be given:
526 1) Task Query
527 2) evaluation_criteria (containing final_answer_metrics / min_tool_calls / must_call_sequence)
528 3) Agent's Final Answer
529 4) Agent's trajectory (sequence of tool calls, possibly including parameters and output summaries)
530
531 Please provide a comprehensive score from 0-100 and explain the reasoning, focusing on:
532 - Whether the final answer covers the key points of final_answer_metrics (clearly state met/missing items)
533 - Whether the trajectory reflects reasonable data acquisition and calculation (avoid "fabrication")
534 - Whether it satisfies the intent of min_tool_calls (quantity is for reference; focus on necessary steps)
535 - must_call_sequence is for "intent sequence" reference: if tool names don't match exactly, judge if the
536 ↪ intent aligns (e.g., News Search -> Data Fetch -> Calculation -> Synthesis)
537
538 Output must be strict JSON (no extra text) in the following format:
539 {
540     "score": 0,
541     "reasoning": "",
542     "met_metrics": [],
543     "missed_metrics": []
544 }
545 }

```

547 D Evaluation Rigor and Methodology Details

548 D.1 Multi-Judge Consistency Analysis

549 To ensure the reliability of LLM-as-a-Judge, we conducted a calibration study between
550 **GPT-5.4** and **Claude-Opus-4.6** across 60+ complex investment research tasks.

Table 5: Multi-judge consistency metrics.

Metric	Value	Interpretation
Pearson Correlation (r)	0.786	Strong positive correlation in scoring trends.
Mean Absolute Difference (MAD)	16.89 / 100	Systemic offset (Claude is more lenient).
Agreement Rate ($\Delta \leq 20$)	66.7%	High consistency in identifying high/low performers.
Agreement Rate ($\Delta \leq 10$)	22.8%	Divergence in fine-grained rubric interpretation.

551 The strong correlation ($r = 0.786$) validates the use of a Dual-Judge ensemble to mitigate
552 single-model bias. While Claude-Opus-4.6 is generally more “optimistic,” both models
553 consistently identify the same performance bottlenecks.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction clearly state our contributions (evolution engine, hierarchical memory, orchestrator) and report the main experimental results on the FinAgent Complex Orchestration Benchmark ($N = 300$).

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We discuss limitations regarding sample size, judge bias, and tool non-determinism in Section 6.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[N/A\]](#)

Justification: This paper is primarily empirical and does not introduce new theoretical theorems or formal proofs.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: We provide full details of the benchmark, tools, and judge prompts in the Appendix and the supplementary code.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: All code, benchmark data, and evaluation scripts are included in the supplementary material for reproducibility.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Full experimental settings, including LLM models, temperatures, and tool configurations, are detailed in Section 4 and the Appendix.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: All main results in Table 3 are reported with 95% confidence intervals across 300 tasks. Figures 3 and 4 show aggregate trends without per-point error bars for visual clarity.

603 **8. Experiments compute resources**

604 Question: For each experiment, does the paper provide sufficient information on the
605 computer resources (type of compute workers, memory, time of execution) needed
606 to reproduce the experiments?

607 Answer: [Yes]

608 Justification: Details of the compute environment and API usage are provided in
609 the Appendix.

610 **9. Code of ethics**

611 Question: Does the research conducted in the paper conform, in every respect, with
612 the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

613 Answer: [Yes]

614 Justification: Our research adheres to the ethics guidelines, specifically regarding
615 data usage and evaluation objectivity.

616 **10. Broader impacts**

617 Question: Does the paper discuss both potential positive societal impacts and
618 negative societal impacts of the work performed?

619 Answer: [Yes]

620 Justification: Potential impacts on financial automation and risks of autonomous
621 trading are discussed in Section 6.

622 **11. Safeguards**

623 Question: Does the paper describe safeguards that have been put in place for
624 responsible release of data or models that have a high risk for misuse (e.g., pre-
625 trained language models, image generators, or scraped datasets)?

626 Answer: [N/A]

627 Justification: Our work does not release high-risk generative models or sensitive
628 datasets.

629 **12. Licenses for existing assets**

630 Question: Are the creators or original owners of assets (e.g., code, data, models),
631 used in the paper, properly credited and are the license and terms of use explicitly
632 mentioned and properly respected?

633 Answer: [Yes]

634 Justification: All third-party tools (Tavily, QVeris) are properly cited and used
635 according to their licenses.

636 **13. New assets**

637 Question: Are new assets introduced in the paper well documented and is the
638 documentation provided alongside the assets?

639 Answer: [Yes]

640 Justification: We introduce a new complex multi-skill orchestration benchmark,
641 which is documented in the Appendix and the supplementary code.

642 **14. Crowdsourcing and research with human subjects**

643 Question: For crowdsourcing experiments and research with human subjects, does
644 the paper include the full text of instructions given to participants and screenshots,
645 if applicable, as well as details about compensation (if any)?

646 Answer: [N/A]

647 Justification: Our research does not involve crowdsourcing or human subjects
648 experiments.

649 **15. Institutional review board (IRB) approvals or equivalent for research**
650 **with human subjects**

651 Question: Does the paper describe potential risks incurred by study participants,
652 whether such risks were disclosed to the subjects, and whether Institutional Review
653 Board (IRB) approvals (or an equivalent approval/review based on the requirements
654 of your country or institution) were obtained?

655 Answer: [N/A]

656 Justification: Our research does not involve human subjects.

657 **16. Declaration of LLM usage**

658 Question: Does the paper describe the usage of LLMs if it is an important, original,
659 or non-standard component of the core methods in this research? Note that if the
660 LLM is used only for writing, editing, or formatting purposes and does *not* impact
661 the core methodology, scientific rigor, or originality of the research, declaration is
662 not required.

663 Answer: [Yes]

664 Justification: LLMs are core components of our method, specifically for the Evolution
665 Engine, DAG Orchestrator, and Dual-Judge Ensemble.